LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL IV dan V
The Core of OOP dan Encapsulation

Disusun Oleh:

Alifa Fatin Ramadhani 105224019

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

A. OOP

OOP (Object-Oriented Programming) atau Pemrograman Berorientasi Objek adalah paradigma pemrograman yang berfokus pada penggunaan objek untuk merancang dan mengimplementasikan program. OOP membantu dalam membuat kode yang lebih terstruktur, mudah dikelola, dan dapat digunakan kembali.

B. Encapsulation

Variabel, konstruktor, dan metode. Dengan menggunakan access modifiers, kita dapat membatasi bagian mana saja dari program yang dapat mengakses suatu member (atribut atau metode) sehingga mendukung prinsip enkapsulasi dan keamanan data pada pemrograman berorientasi objek. Ada 4 jenis access modifiers dalam Java, yakni:

1. Default (tidak diperlukan kata kunci)
2. Private
3. Protected
4. Public

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	nomorKamar	String	Menyimpan data nomor kamar secara private
2	tipeKamar	String	Menyimpan tipe kamar secara private
3	kapasitasMaksimal	int	menentukan kapasitas maksimal kamar
4	hargaPerMalam	double	Menyimpan harga sewa per malamnya
5	isTersedia	boolean	Menentukan apakah kamar tersedia atau tidak

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1.	`KamarHotel(String nomorKamar, String tipeKamar, int`	Constructor	Untuk pendaftaran kilat yang hanya menerima parameter nomor kamar, tipe kamar,

	`kapasitasMaksimal)`		dan kapasitas yang tersedia
2	`KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerMalam)`	Constructor	Untuk menyimpan atau menginsialisasikan data kamar secara lengkap
	`String getNomorKamar()`	Functional	Mengambil nomorKamar pada variabel nomorKamar
	`String getTipeKamar()`	Functional	mengambil tipe kamar pada variabel tipeKamar
	`int getKapasitasMaksimal()`	Functional	mengambil kapasitas maksimal pada variabel kapasitasMaksimal
	`double getHargaPerMalam()`	Functional	mengambil harga pada variabel hargaPerMalam
	`boolean isTersedia()`	Functional	mengambil status kamar nantinya
	`void setTipeKamar(String tipeKamar)`	Procedural	Untuk menentukan tipe kamar apa saja nantinya yang akan di simpan
	`void setKapasitasMaksimal(int kapasitasMaksimal)`	Procedural	Untuk mengatur kapasitas kamar

	`void setHargaPerMalam(double hargaPerMalam)`	Procedural	Untuk mengatur harga kamar hotel per malamnya
	`void setIsTersedia(boolean isTersedia)`	Procedural	untuk mengatur ketersediaan kamar hotel
	`void pesanKamar()`	Procedural	Untuk memesan kamar tanpa parameter
	`esanKamar (int jumlahTamu)`	Procedural	Memesan kamar dengan validasi jumlah tamu
	`void batalkanPesanan()`	Procedural	Untuk membatalkan pesanan
	`double hitungTotalBayar(int jumlahMalam)`	Functional	Menghitung total bayar berdasarkan jumlah malam
	`double hitungTotalBayar(int jumlahMalam, String kodeVoucher)`	Functional	Menghitung total bayar berdasarkan jumlah malam dan diskon voucher
	`void tampilkanProfilKamar()`	Functional	Untuk mencetak display sesuai dengan perintah

IV. Dokumentasi dan Pembahasan Code

A. KamarHotel.java

```
Windsurf: Refactor | Explain
public class KamarHotel {
    //atribut krusial
    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean isTersedia;
```

- Sesuai dengan soal yang diberikan, bahwasannya atribut dinilai private karena krusial atau tidak bisa di akses begitu saja.

```
public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal){
    this.nomorKamar = nomorKamar;
    this.tipeKamar = tipeKamar;
    this.kapasitasMaksimal = kapasitasMaksimal;
    this.hargaPerMalam = 0;
    this.isTersedia = true;
  }
```

- Merupakan konstruktor yang digunakan untuk menginisialisasi objek pada KamarHotel berdasarkan data-data (menyesuaikan atribut)

```
public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerMalam){
    this.nomorKamar = nomorKamar;
    this.tipeKamar = tipeKamar;
    this.kapasitasMaksimal = kapasitasMaksimal;
    this.hargaPerMalam = hargaPerMalam;
    this.isTersedia = true;
  }
```

- Konstruktor parameter lengkap yang digunakan untuk menginisialisasi seluruh atribut utama pada KamarHotel termasuk hargaPerMalam. Perdedaan dengan konstruktor sebelumnya adalah, pada konstruktor ini memungkinkan user untuk langsung memasukkan nilai hargaPerMalam ketika object dibuat sehingga tidak memerlukan pengisian ulang pada setter

```java
//getter untuk setiap atribut
Windsurf: Refactor | Explain | ×
public String getNomorKamar() {
        return nomorKamar;
    }

Windsurf: Refactor | Explain | Generate Javadoc | ×
public String getTipeKamar() {
        return tipeKamar;
    }

Windsurf: Refactor | Explain | Generate Javadoc | ×
public int getKapasitasMaksimal() {
        return kapasitasMaksimal;
    }

Windsurf: Refactor | Explain | Generate Javadoc | ×
public double getHargaPerMalam() {
        return hargaPerMalam;
    }

Windsurf: Refactor | Explain | Generate Javadoc | ×
public boolean isTersedia() {
        return isTersedia;
    }
```

- Membuat getter untuk setiap atribut, fungsinya untuk mengambil nilai pada atribut tanpa mengubahnya, i memungkinkan akses baca terhadap atribut yang bersifat private, sehingga data tetap terlindungi dari akses langsung.

```java
public void setTipeKamar(String tipeKamar){
              if(tipeKamar.equalsIgnoreCase("Reguler")     ||
tipeKamar.equalsIgnoreCase("Premium")                      ||
tipeKamar.equalsIgnoreCase("Suite")){
      this.tipeKamar = tipeKamar;
   } else {
            System.out.println("peringatan:  tipe  kamar"  +
tipeKamar + "tidak valid. maka tipe akmar akan dipaksanakan
menjaid reguler");
      this.tipeKamar = "Reguler";
   }
 }
```

- Maksud code di atas adalah Setter dengan validasi yang fungsinya mengubah nilai atribut
- Pada kasus ini setTipeKamar(), hanya menerima tiga tipe yaitu *Reguler*, *Premium* , dan *Suite*

```
public void setKapasitasMaksimal(int kapasitasMaksimal) {
        this.kapasitasMaksimal = kapasitasMaksimal;
    }

public void setHargaPerMalam(double hargaPerMalam){
   if (hargaPerMalam < 50000){
       System.out.println("peringatan: harga per malam tidak
boleh kurang dari 50000. maka harga per malam akan
dipaksakan menjadi 50000");
      this.hargaPerMalam = 50000;
   } else {
      this.hargaPerMalam = hargaPerMalam;
   }
}
```

- pada "public void setKapasitasMaksimal(int kapasitasMaksimal)" digunakan untuk mengubah nilai pada kapasitas maksimal kamar, dimana ini tidak akan ada validasi
- pada "void setHargaPerMalam(double hargaPerMalam)" dimana setter mempunyai validasi yaitu ketika harga dibawah 50000 maka akan di tolak dan dipaksanakn menjadi 50000. Namun jika tidak dibawah 50000 maka akan diterima

```
public void setIsTersedia(boolean isTersedia) {
        this.isTersedia = isTersedia;
    }

//method overloading pesan kamar
public void pesanKamar(){
   if(isTersedia){
      isTersedia = false;
       System.out.println("Kamar " + nomorKamar + " berhasil
dipesan.");
        } else {
                System.out.println("peringatan: Kamar " +
nomorKamar + " sudah terisi!");
   }
}
public void pesanKamar (int jumlahTamu){
   if(!isTersedia){
      System.out.println("peringatan: Kamar " + nomorKamar +
" sudah terisi!");
   } else if (jumlahTamu > kapasitasMaksimal){
```

```
            System.out.println("peringatan:  jumlah  tamu"  +
jumlahTamu  +  "melebihi  kapasitas  maksimal  kamar  "  +
nomorKamar + "!");
    } else {
        isTersedia = false;
         System.out.println("Kamar " + nomorKamar + " berhasil
dipesan untuk " + jumlahTamu + " tamu.");
    }
}
```

- Method “setTersedia()” merupakan sebuah setter yang dipakai untuk mengubah status sedianya kamar, nilai boolean akan langsung disimpan ke dalam atribut tanpa validasi apapun.
- Sedangkan methode “pesanKamar()” digunakan untuk melakukan proses pemesanan kamar, namun mengecek ketersediaan kamar dulu.
- Method “pesanKamar(int jumlahTamu)” merupakan method yang digunakan untuk melakukan pemesanan kamar dengan melihat jumlah tamu, dengan memeriksa apakah tamu tersedia, jika tidak tersedia akan menampilkan peringatan jika namun jika tersedia maka data akan langsung diinput.

```
public void batalkanPesanan(){
    isTersedia = true;
     System.out.println("Pesanan untuk kamar " + nomorKamar +
" telah dibatalkan.");
}

//methode overloading untuk sistem penagihan

public double hitungTotalBayar(int jumlahMalam){
    return hargaPerMalam * jumlahMalam;
}

public  double  hitungTotalBayar(int  jumlahMalam,  String
kodeVoucher){
    double total = hargaPerMalam * jumlahMalam;
     if  (kodeVoucher.equalsIgnoreCase("PROMO")  &&  jumlahMalam
>= 3)  {
            double diskon = total * 0.20;
             System.out.println("Voucher P R O M O berhasil!
Diskon 20% = Rp" + diskon);
            return total - diskon;
        } else {
```

```
                System.out.println("Kode voucher tidak valid
atau tidak memenuhi syarat untuk diskon.");
            }
            return total;
}
```

- Method batalkan pesanan digunakan untuk membatalkan pesanan kamar dengan mengubah ketersediaan kamar menjadi tersedia
- Method hitung total bayar dengan parameter jumlah malam berfungsi untuk menghitung total biaya menginap berdasarkan jumlah malam tanpa adanya diskon
- Method hitung total bayar dengan parameter jumlah malam dan voucher merupakan method overloading dimana memungkinkan perhitungan total biaya dengan tambahan fitur voucher

```
125
126   //tampilan
127
        Windsurf: Refactor | Explain | Generate Javadoc | X
128   public void tampilkanProfilKamar() {
129           System.out.println(x: "============================");
130           System.out.println(x: " kamar hotel profile ");
131           System.out.println(x: "============================");
132           System.out.println("No Kamar  : " + nomorKamar);
133           System.out.println("Tipe Kamar   : " + tipeKamar);
134           System.out.println("kapasitas    : " + kapasitasMaksimal + " orang");
135           System.out.println("harga per malam : Rp" + hargaPerMalam);
136           System.out.println("Status: " + (isTersedia ? "Tersedia" : "Sudah Dipesan"));
137           System.out.println(x: "============================");
138       }
139
140   }
141
```

- Tampilan atau display untuk output ketika user sudah memasukkan data-data.

B. Main.java

```
System.out.println("==uji coba kamar 1==");
                KamarHotel  kamar1  =  new  KamarHotel("100",
"Presidential", 3 );
            System.out.println("==uji coba kamar 1 dengan harga
-10000==");
        kamar1.setHargaPerMalam(-10000);
```

```
  System.out.println("==uji coba kamar 2==");
                KamarHotel kamar2 = new KamarHotel ("200",
"Premium", 2, 150000);
         System.out.println("==uji coba pesan kamar 2 dengan
kapasitas 4 orang==");
         kamar2.pesanKamar(4); //ini harusnya kan di tolak
         System.out.println("==uji coba pesan kamar 2 dengan
kapasitas 2 orang==");
          kamar2.pesanKamar(2); //ini harusnya kan di terima
ea ea
         System.out.println("==uji coba pesan kamar 2 dengan
kapasitas 2 orang==");
          kamar2.pesanKamar(2); //ini harusnya kan di terima
ea ea
System.out.println("== tagihan kamar 1 untuk 2 malam dengan
voucher PROMO==");
        double totalBayarKamar1 = kamar1.hitungTotalBayar(2,
"PROMO");
            System.out.println("Total bayar kamar 1: Rp" +
totalBayarKamar1);

        System.out.println("== tagihan kamar 2 untuk 4 malam
dengan voucher PROMO==");
        double totalBayarKamar2 = kamar2.hitungTotalBayar(4,
"PROMO");
            System.out.println("Total bayar kamar 2: Rp" +
totalBayarKamar2);

        System.out.println("==================");
        System.out.println("Struk Informasi");
        System.out.println("==================");
        kamar1.tampilkanProfilKamar();
        System.out.println();
        kamar2.tampilkanProfilKamar();
```

- Untuk uji coba

```
==uji coba pesan kamar 2 dengan kapasitas 2 orang==
peringatan: Kamar 200 sudah terisi!
== tagihan kamar 1 untuk 2 malam dengan voucher PROMO==
Kode voucher tidak valid atau tidak memenuhi syarat untuk diskon.
Total bayar kamar 1: Rp100000.0
== tagihan kamar 2 untuk 4 malam dengan voucher PROMO==
Voucher P R O M O berhasil! Diskon 20% = Rp120000.0
Total bayar kamar 2: Rp480000.0
===================
Struk Informasi
===================
==============================
 kamar hotel profile
==============================
No Kamar  : 100
Tipe Kamar   : Presidential
kapasitas    : 3 orang
harga per malam : Rp50000.0
Status: Tersedia
==============================

==============================
 kamar hotel profile
==============================
No Kamar  : 200
Tipe Kamar   : Premium
kapasitas    : 2 orang
harga per malam : Rp150000.0
Status: Sudah Dipesan
==============================
PS D:\Prak PBO\Modul5\ThtPraktikum\src>
```

- Maka sebelumnya, pada code main.java diperlihatkan ketika tipe kamar diubah menjadi Presidential maka tidak valid.
- Ketika kamar diberikan harga -10000 maka tidak akan berhasil karena bukan nilainya dan akan menetapkan ke harga 50000.
- Awalnya ditetapkan kapasitas 2 orang namun setelah itu dilakukan uji coba ketika 4 orang maka akan ditolak namun ketika 2 orang akan diterima.
- Begitu juga dengan adanya voucher PROMO maka akan dihitung total setelah menggunakan voucher PROMO.

V. Kesimpulan

Berdasarkan hasil implementasi dan melakukan uji program dapat disimpulkan OOP khususnya encapsulation telah dibuat dengan baik pada class kamarHotel. Seluruh atribut penting bisa dibuat menjadi private sehingga tidak bisa di akses oleh siapapun.

VI. Daftar Pustaka